Clothes Segmentation for Personal Photos

SegFormer-B3 fine-tuned on ATR: data, model selection, evaluation and limits

Eyad Elnakib

***Abstract*—This paper describes a system that segments the clothes worn by a person in a personal photo. It returns a per-pixel label mask, a colour overlay and one transparent cut-out per garment. The model is a SegFormer-B3 network (47.2 M parameters) fine-tuned on the ATR human-parsing dataset with 15 classes: 10 garment and accessory classes, plus background, hair, face, arms and legs.**

Method. More than a dozen datasets and about twenty models were considered. Most were excluded for documented reasons: wrong labels, too large for one 22 GB GPU, too slow to train in the time available, or not openly available. Five architectures were then trained under identical conditions, with a selection rule fixed before the runs; SegFormer-B3 won.

Results. On a held-out ATR test split (960 photos, evaluated once), the final model reaches mIoU 0.789, garment mIoU 0.744 and clothing-against-rest IoU 0.930. On 986 photos from a second dataset it never saw (CCP), garment mIoU is 0.677. On those photos it scores above YOLO26x-sem trained here on the same data, and above two ready-made parsers (SCHP, FASHN). An error analysis explains why each scores lower.

Cost and limits. The whole project used about 46 GPU hours on one NVIDIA L4. Of twelve changes tried after selection, only longer training and flip averaging met their pre-registered rules. The main weakness is the garment type in ambiguous cases (dress against top with skirt), then thin items (belt, scarf, sunglasses). The next improvements identified were stopped by compute and time, not by a lack of ideas.

***Index Terms*—clothes segmentation, human parsing, semantic segmentation, SegFormer, ATR, cross-dataset evaluation**

I. Datasets

The task needs a pixel-level label map that separates garments by type (top, skirt, pants, dress, …) and also separates them from skin and hair. Skin and hair are what a garment is most easily confused with, and a try-on system must leave them untouched. Each candidate was judged on four questions:

1) Does it label what the task needs?
2) Can it be obtained?
3) Can it be trained on with one GPU in the time available?
4) Is the label quality usable?
5) How long would the model need to process it, that is, to train on it on our one GPU?

Numbers for downloaded and audited datasets are measured; the others come from the cited source.

A. Datasets considered and not chosen

Table I lists every dataset discussed. The main reason for rejecting each falls into one of five groups:

- **Labels:** the classes do not fit the task;
- **Quality:** the labels or photos are not good enough;
- **Access:** closed, gated or no longer downloadable;
- **Compute/time:** too large, or a fair test would not fit the budget or deadline;
- **Licence.**

B. Chosen dataset: ATR

ATR [1] has 17,706 photos of single people, mostly full-body fashion shots, each with an 18-class pixel label map; it was loaded from a Hugging Face copy [2]. It was chosen because:

- **its classes fit the task:** garment types plus hair, face, arms and legs;
- **one person per photo,** as in a personal photo;
- **the photos match the use case:** 99.8% portrait, 91% exactly 400×600;
- **it is open and fits the budget:** about 800 MB, and 30 epochs train in 5.6 h on one L4.

a) Audit.: No decode errors, size mismatches or invalid labels. 18 byte-identical duplicates were dropped. 54 near-duplicate groups (110 images, 64-bit difference hash) spanned splits, and their validation and test members were moved to training, because such copies inflate test scores [3]. Splits (seed 42): **16,038 / 690 / 960**, with the published split sizes [4]. The official split could not be used: the copy holds only an image and a mask per photo, with no file names.

b) Classes.: Left and right shoes, legs and arms were merged into 15 classes. Only shoes among them are clothing, and merging makes mirror augmentation safe without relabelling. This is a judgement; 18 classes were not compared. The classes are very unbalanced: background 77% of pixels, belt 0.06%, scarf 0.17%. ATR has no coat class and usually labels tights as pants.

c) Preprocessing and augmentation.: Photos are resized to fit 576×384, cached, and placed on a canvas whose padding carries the ignore label. Training uses:

- flip ($p = 0.5$);
- scale 0.5–1.5 with random placement [5];
- rotation ±15°;
- mild colour changes;
- blur, noise and JPEG compression ($p = 0.15$–0.2).

C. Dataset used only for testing

CCP [6]: 1,004 street photos, 59 classes, Apache-2.0. Labels were mapped to 13 classes. 18 photos within 4 hash bits of ATR were removed, leaving **986**. It is the only outside set with garment types plus skin and hair.

TABLE I

DATASETS CONSIDERED AND WHY THEY WERE NOT USED FOR TRAINING. THE MAIN REASON IS IN BOLD, FOLLOWED BY THE DETAILS.

Dataset	Size and labels	Why not used (main reason, then details)
LIP [7]	30,462 train, 10,000 val; 20 classes	**Quality, labels.** Audited locally (40,462 pairs): small photos (median 210×318), bodies cut off (34% show head and legs), skirt in 2.1% and dress in 3.9% of photos (ATR: 28%, 26%), no bag or belt; reported wrong labels and many minors [8].
CIHP [9]	28,280 train, 5,000 val; 20 classes	**Quality, labels.** Audited: 2–12 people per photo (task: one person); 177 photos annotated twice agree at only 0.79 mIoU; 83 identical-photo groups across splits; no belt or bag.
MHP v2.0 [10]	25,403 photos; 58–59 classes	**Labels, time.** 2–26 people per photo, small people; each would have to be cropped and remapped. No licence found.
DeepFashion2 [11]	491,895 photos; 13 garment classes	**Access, labels.** Password requested by form; garments only, as polygons; no skin, hair or face.
ModaNet [12]	55,176 photos; 13 garment classes	**Access.** The repository holds only annotations; its photo source (PaperDoll, `photos.lmdb.tar`, about 40 GB) now redirects to a lab home page (checked 17 Oct 2026). Garments only; scarf and tie merged; CC BY-NC.
Fashionpedia / iMaterialist [13]	about 48,800 photos; 27 garment classes + 19 parts	**Labels.** Garments only, no skin, hair or face; many close-ups. Annotations CC BY 4.0; photos keep their own licences [14].
DeepFashion-MultiModal [20]	44,096 photos (5.4 GB); 12,701 with 24-class parsing	**Compute/time, licence.** Labels downloaded and counted: scarf in 48 of 12,701 masks, belt 717, eyeglasses 527, so it cannot fix our weakest classes. Studio photos only, 88% women, 6,902 distinct items. A fair test (partial-label loss, fresh control, statistics) needs 6–10 h of GPU and coding. Non-commercial research only.
SHHQ [15]	40,000 full-body photos	**Quality, access.** Masks made by a model, not people; access agreement.
VITON-HD / DressCode maps [16]	–	**Quality.** Parsing maps are another model's (SCHP) output.
4D-DRESS [17]	78k scans, 64 outfits	**Access, labels.** 3D scans, little variety; research agreement.
SynBody, BEDLAM [18]	up to 1.2 M rendered images	**Compute/time, quality.** Very large; no evidence that synthetic garment masks help on real photos.
Pascal-Person-Part [19]	7 body-part classes	**Labels.** Body parts only; a parsing benchmark.
Cleaned/relabelled ATR	–	**Access.** None found.

II. MODELS

About twenty models were considered, on one Colab Pro account with an NVIDIA L4 (22.5 GB).

A. Models not used, and why

Table II lists them. Three are simply too large for one 22 GB GPU and the deadline. The others are impractical, use the wrong task formulation, or have no published size or weights. Figure 1 puts their sizes next to the models trained here.

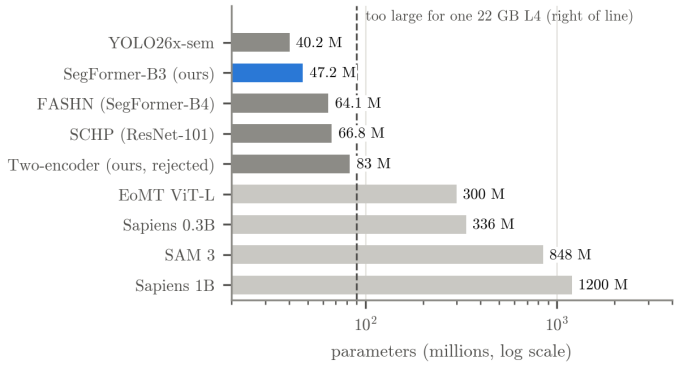


Fig. 1. Model sizes on a log scale. Our SegFormer-B3 in blue; models trained or compared here in grey; models judged too large for one 22 GB L4 in light grey (Table II).

B. Models trained at 5 and 12 epochs

Five architectures were trained under identical conditions: same data, augmentation, loss, seed, batch 16 and schedule. The rule was fixed before the runs: *the highest validation garment mIoU wins; within 1 point the faster model wins; a collapse on a core garment is a veto.* Round 1 stopped at epoch 5 of a 12-epoch schedule; the best three continued to epoch 12. Variants of the winner and YOLO26x-sem were later trained for 12 epochs under fixed rules too (Table III); the round-1 curves are in Figure 2. Points to note:

- The order changed between epoch 5 and epoch 12, so a 5-epoch comparison alone would have picked a different model.
- The finalists are within 1.2 points on one seed, and the larger B4 did not beat B3.
- All models used SegFormer's optimiser settings, which may disadvantage the convolutional ones.

C. Why SegFormer-B3

- It won the controlled comparison at epoch 12 (Table III).
- No variant or alternative beat it (Section IV-C).
- It fits the hardware: 14.4 GB to train at batch 16, and 56 ms per photo.
- It accepts any input size.

UPerNet + ConvNeXt-Tiny is a reasonable faster alternative: 1.2 points lower, 8.3 GB to train, and about a third of the inference time. Figure 3 shows where B3's lead comes from: skirt and dress, the two hardest core garments.

TABLE II
MODELS CONSIDERED BUT NOT USED, WITH THE PROBLEM THAT RULED EACH OUT.

Model	Size	Problem: why it was not used
EoMT, ViT-L (DINOv2/v3) [21]	≈300 M (6×)	**Too large.** May not fit 22 GB at 576×384, batch 16; each run several times our 5.6 h; overfitting risk on 16k photos (seen with our 83 M model); DINOv3 weights gated.
Sapiens 0.3B–1B [22]	336 M–1.2 B	**Too large, wrong classes.** 1,024 px native, 7–25× our size; only "upper/lower clothing", so a new head is needed; non-commercial licence.
SAM 3 [23]	848 M	**Too large, wrong output.** One mask per text prompt, not one of 15 classes per pixel.
SCHP training [19]	66.8 M	**Too slow to reproduce.** 150 epochs plus self-correction cycles in an old code base. Its ready-made weights were compared instead (Section IV-C).
SegNeXt [24]	B3-size	**Code not usable.** Official code needs an old MMSegmentation; strongest untested candidate.
HRNet + OCR [25]	–	**Weaker evidence.** 56.65 on LIP, below SegFormer-class models; omitted at first.
Mask2Former / M2FP [26]	–	**Slow, licence.** Converges slowly, aims at instance separation; M2FP weights CC BY-NC (59.86 on LIP).
YOLO instance seg. [27]	–	**Wrong task.** One box and mask per object, not a label per pixel; ATR maps break into objects. (The semantic YOLO26-sem fits and was tested.)
SCHNet, SAM-based small-device parser, UniHCP, HumanBench, Spectrum, CDGNet [28]	not published	**No size or weights.** Memory and time on the L4 cannot be judged; no packaged model for these classes; results on LIP, not this task.

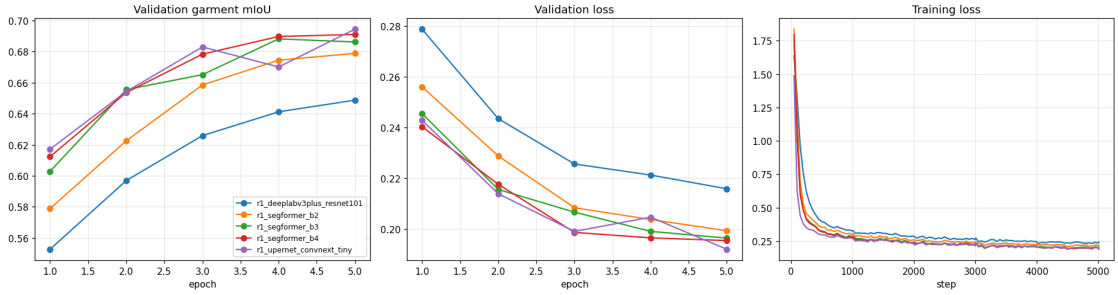


Fig. 2. Round 1 of the bake-off (5 epochs, 12-epoch schedule): validation garment mIoU and validation loss per epoch, and training loss per step (about 1,000 steps per epoch), for the five models.

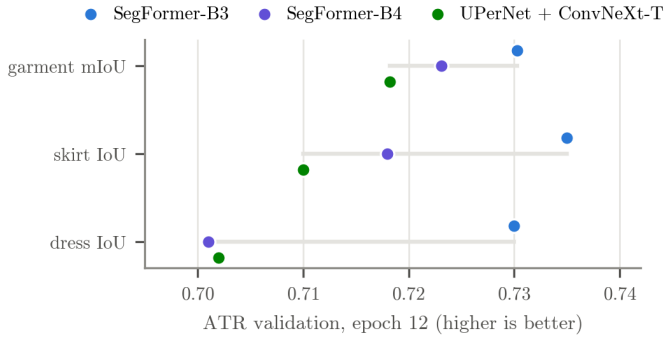


Fig. 3. Bake-off round 2 (epoch 12, ATR validation): the three finalists on garment mIoU, skirt and dress (Table III).

III. METHOD

A. Architecture

SegFormer [29] pairs a hierarchical transformer encoder (MiT) with a light all-MLP decoder (Figure 4). The B3 settings below were read from the model's own configuration.

- **Encoder:** four stages of 3, 4, 18 and 3 blocks, 64, 128, 320 and 512 channels wide, at 1/4, 1/8, 1/16 and 1/32 of the input.
- **Patch embeddings** overlap: 7×7 with stride 4, then 3×3 with stride 2.
- **Attention** is efficient self-attention: keys and values are reduced by 8, 4, 2 and 1, with 1, 2, 5 and 8 heads.
- **Mix-FFN** uses a 3×3 depth-wise convolution, which gives position information, so there is no fixed positional encoding and any input size works.
- **Decoder:** each stage is projected to 768 channels, upsampled to 1/4, concatenated, fused by a 1×1 convolution with BatchNorm and ReLU, classified into 15 classes, and upsampled ×4.

Encoder 44.1 M parameters (ImageNet, `nvidia/mit-b3`); decoder 3.2 M (random start); 47.2 M in total.

B. Loss function

The loss is $\mathcal{L} = 0.5\,\mathcal{L}_{\text{CE}} + 0.5\,\mathcal{L}_{\text{Lovász}}$, computed in fp32. Table IV compares it with the other candidates.
The combination was chosen because:

- cross-entropy gives stable optimisation;
- Lovász optimises the evaluation metric per class, so a belt (0.06% of pixels) counts as much as trousers;
- the 0.5/0.5 mix is SCHP's segmentation loss [30], the best published method on ATR.

It was not ablated against plain cross-entropy.

TABLE III

EVERY MODEL TRAINED FOR 5 OR 12 EPOCHS ON ATR. VALIDATION (690 PHOTOS) AT 576×384 UNLESS MARKED; ONE SEED. GPU = TRAINING TIME ON THE L4 UP TO THAT EPOCH, PLUS ABOUT 50 MIN SETUP.

Model	Epochs	Garment mIoU	mIoU	GPU	Outcome
Bake-off, round 1					
UPerNet + ConvNeXt-Tiny	5	0.6945	0.7510	1.5 h	To round 2
SegFormer-B4	5	0.6911	0.7469	2.0 h†	To round 2
SegFormer-B3	5	0.6862	0.7432	1.8 h†	To round 2 (tie with B4, faster)
SegFormer-B2	5	0.6789	0.7378	1.6 h†	Stopped
DeepLabV3+ (ResNet-101)	5	0.6487	0.7144	1.5 h†	Stopped (4.6 points behind)
Bake-off, round 2					
SegFormer-B3	12	**0.7303**	**0.7762**	3.1 h	**Chosen**
SegFormer-B4	12	0.7231	0.7718	3.6 h†	Not better than B3
UPerNet + ConvNeXt-Tiny	12	0.7182	0.7699	2.5 h	Third; the faster alternative
Variants of SegFormer-B3 (rule: at least +0.5 and no core garment −1)					
B3 + edge branch [5]	12	0.7212	0.7707	3.5 h	Rejected
B3 from ADE20K weights	12	0.7145	–	3.1 h†	Rejected
B3, half learning rate	12	0.7179	–	3.1 h†	Rejected
B3, double learning rate	12	0.7348	–	3.1 h†	Rejected (+0.46; dress −1.1)
B3 + ConvNeXt-T two-encoder, 83 M [31]	12 (+5)	0.7315	0.7812	4.1 h + 2.2 h†	Rejected (dress −2.4; fell to 0.7067 by epoch 17)
Another architecture, trained on the same ATR photos					
YOLO26x-sem [32] (original size)	12	0.6895	0.7458	5.8 h†	Rejected; B3 at 12 epochs, original size: 0.7316 / 0.7779

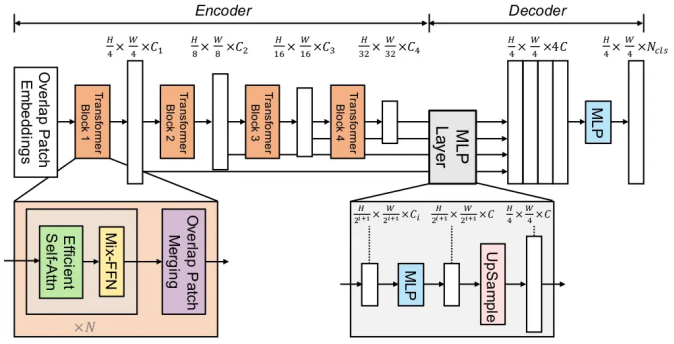


Fig. 4. SegFormer architecture, from the SegFormer paper [29]. For our B3: $N = 3, 4, 18, 3$ blocks and C_1–$C_4 = 64, 128, 320, 512$ channels in the four stages, decoder width $C = 768$, and $N_{cls} = 15$ classes; the $H/4 \times W/4$ output is upsampled ×4 to the input size.

TABLE IV

LOSS CANDIDATES.

Loss	Class imbalance	Note
Cross-entropy	Large classes dominate	Stable
Weighted CE	Better rare recall	Weight-sensitive; bleeding
Focal	Down-weights easy pixels	One more setting
Dice	Per class	Unstable if class absent
Lovász-Softmax [33]	Optimises per-class IoU	Noisy alone early

C. Training

Table V lists the training settings.

TABLE V

TRAINING SETTINGS (NONE TUNED ON ATR).

Setting	Value	Basis
Input	576×384	ATR photo shape; FASHN [8]
Optimiser	AdamW, β=(0.9, 0.999), wd 0.01	SegFormer config [34]
Learning rate	$6{\cdot}10^{-5}$ enc., $6{\cdot}10^{-4}$ dec.	Same (head ×10)
Schedule	1,500 warm-up steps, linear decay	Same
Batch, precision	16; fp16 (loss fp32)	Same
Epochs	30 (30,060 steps)	From 12-epoch curves
GPU	L4 22 GB: 5.6 h training + about 50 min setup = 6.4 h	Training measured (336 min); setup estimated

a) Final run.: Validation mIoU rose from 0.676 after epoch 1 to 0.787 at epoch 28, the checkpoint kept. The curve is flat from epoch 21, and the validation loss stops falling at about epoch 17 without rising, so there is no sign of overfitting. Compared with 12 epochs, 30 epochs added 1.0 point of mIoU, mostly on rare classes: scarf +5.7, belt +2.6.

b) Robust to interrupted sessions.: Colab sessions can end at any time, so:

- the full state is saved every 10 minutes and after each epoch, written atomically;
- the data order and augmentation are derived from (seed, epoch, position);
- a kill-and-resume test checks a resumed run step by step against an uninterrupted one.

D. Pipeline and outputs

For a photo of any size, the pipeline (`src/clothseg/inference.py`, `scripts/segment.py`) runs these steps:

1) Read the photo with its EXIF rotation; transparent, grey-scale and CMYK images are converted to RGB.
2) Letterbox it into 576×384 and normalise it.
3) Predict in fp16, falling back to fp32 on overflow; optionally also predict the mirrored photo and average the two (flip averaging).
4) Crop the padding, resize the scores to the photo size (long side at most 1,536 px), and take the arg-max.
5) Write a palette PNG mask, a colour overlay, an RGBA cut-out per garment covering at least 0.2% of the photo, and a JSON summary.

A FastAPI server and a React web demo were also built.

IV. Evaluation

A. Metrics, and why each is used

- *IoU per class* = $\frac{TP}{TP+FP+FN}$ over all pixels of a split: it penalises both missed and added pixels.
- *mIoU*: the plain class mean, so a belt counts as much as trousers.
- *Garment mIoU* (10 classes): the score that matters for the task, since body classes are easy.
- *Core-garment mIoU* (upper-clothes, pants, skirt, dress): the garments try-on edits most.
- *Clothing binary IoU*: clothes against everything else.
- *Boundary IoU* [35]: pixels near contours, which decide cut-out quality.
- *Dice, recall and pixel accuracy*: given for reference only. Pixel accuracy is misleading when 77% of the pixels are background.

B. Performance analysis

Table VI gives the results of the final model.

TABLE VI
FINAL MODEL. FLIP = AVERAGED WITH THE MIRRORED PHOTO (ADOPTED: +0.36 MIOU ON VALIDATION; THRESHOLD 0.2). EVALUATION OF VAL + TEST: ABOUT 1.0 GPU H†.

	Validation		Test	
Metric	plain	flip	plain	flip
mIoU (15)	.7853	.7888	.7859	**.7886**
Garment mIoU	.7394	.7441	.7406	**.7439**
Core-garment mIoU	.7794	.7904	.7825	**.7865**
Clothing binary IoU	.9290	.9296	.9293	**.9298**
Pixel accuracy	.9713	.9720	.9696	**.9701**
Mean Dice	.8762	.8785	.8760	**.8779**
Boundary mIoU	.7313	.7349	.7272	**.7305**
ms / photo (L4)	57	104	56	105

Validation and test agree within 0.1 point. Per class (test, flip), from strongest to weakest (Figure 5):

- 0.85–0.89: legs .889, face .858, pants .848;
- 0.79–0.83: arms .830, upper-clothes .828, hair .827, bag .818, hat .789;
- 0.69–0.77: shoes .770, skirt .746, dress .724, scarf .704, sunglasses .688;
- belt .524, the weakest.

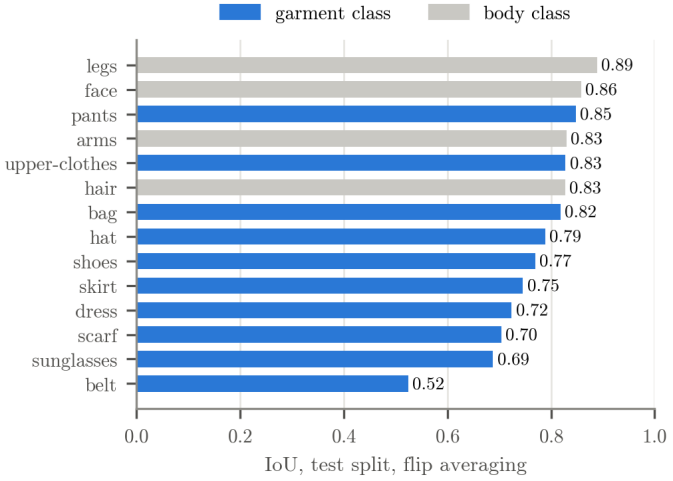


Fig. 5. Per-class IoU of the final model on the test split, with flip averaging. Garment classes in blue, body classes in grey.

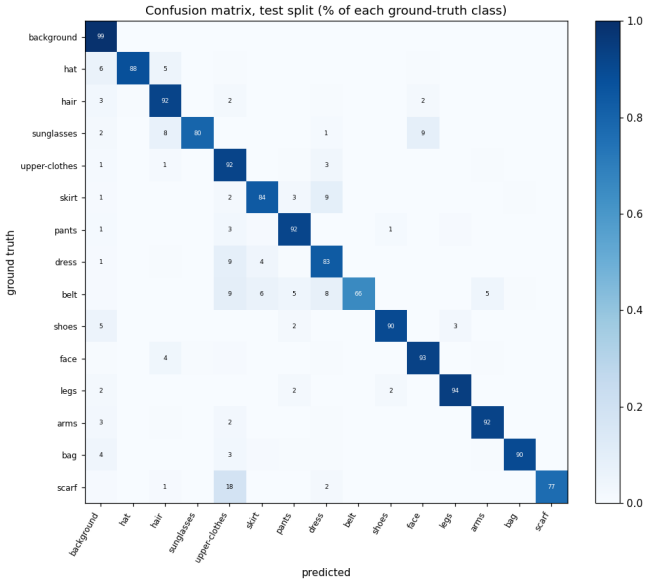


Fig. 6. Confusion matrix on the test split (rows: true class).

The confusion matrix (Figure 6) shows two error patterns.

1) **Garment-type confusion.** Skirt goes to dress 9% of the time, and dress to upper-clothes 9%. So dress and skirt have a low Boundary IoU (0.59, 0.62): a whole garment with the wrong class makes its entire outline wrong.
2) **Thin items absorbed by a neighbour.** 18% of scarf goes to upper-clothes; belt spreads over the garments around it; sunglasses go to face (9%) or hair (8%).

a) Outside ATR (CCP, with flip averaging).:

- garment mIoU 0.677;
- mIoU over 13 classes 0.727;
- clothing binary IoU 0.947.

Before training, the same network scores 0.007, so the result comes from training (Figure 7). Separating clothes from the rest carries over fully; garment *types* do not, with skirt at 0.39 and dress at 0.47. Partly this is a labelling difference: a top over a draped skirt can be "dress" in CCP.

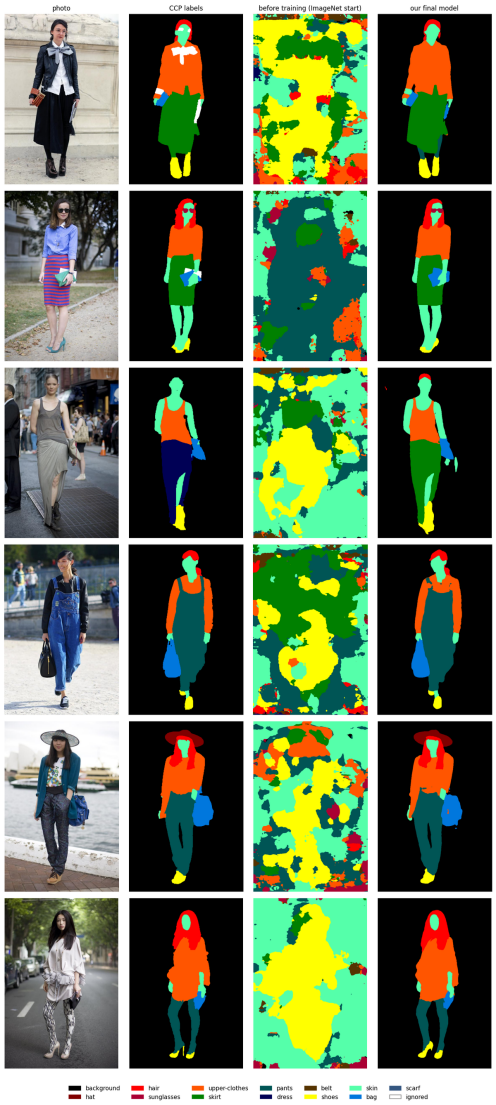


Fig. 7. Six CCP photos: the photo, the CCP labels, the network before training (ImageNet start), and our final model.

C. Comparison with other models

Four models were compared with the final model (30 epochs) on the same 986 CCP photos, with the same mapping and no flip averaging:

- the same network after 12 epochs;
- **YOLO26x-sem** [32] (40.2 M), trained here on the same ATR photos;
- the ready-made **SCHP-ATR** [36] (ResNet-101, 66.8 M);
- the ready-made **FASHN** parser [37] (SegFormer-B4, 64.1 M).

FASHN has no shoes class, so a 9-garment score without shoes is its fair score.

Table VII shows that none is better by the pre-registered rule (+0.5 points and no core garment −1). On ATR validation, with identical training data and 12 epochs, YOLO26x-sem scores 0.690 against 0.732 garment mIoU. It is lower on every class, but 3.6× faster (25 against 91 ms). Figure 9 shows the same comparison garment by garment.

a) Why each one scores lower (measured).: Each model's errors were broken down by where garment pixels went, outlines against insides, garment size, and garments missed or invented.

- **YOLO26x-sem: coarser output.**
  - Its head predicts at 1/8 of the input (Ultralytics 8.4.160); SegFormer predicts at 1/4 [29].
  - Its outline loss is about twice its inside loss: −3.7 against −1.7 on CCP, −3.3 against −1.9 on ATR.
  - Scarf goes to upper-clothes 30% (ours 21%), and hat to hair 14% (ours 5%).
  - Figure 10 shows its worst photos: long skirts and dresses split into skirt, dress and pants patches, and a scarf appears on a coat.
  - The same pattern on identical training data points to the architecture. That is not proof: its Cityscapes start and Dice loss also differ.
- **SCHP-ATR: wrong garment types.**
  - It loses more inside regions (−4.4) than along outlines (−3.4).
  - It invents pants on legs in 41.5% of photos without pants (ours 10.8%).
  - It takes sunglasses for skin (15% against 9%).
- **FASHN: small accessories and different rules.**
  - It is lower on sunglasses (−9.9), hat (−4.8) and belt (−3.1).
  - It *beats* ours on dress and scarf.

Figure 11 shows the error breakdown behind these points, and Figure 12 the photos where SCHP-ATR and FASHN fall furthest behind.

b) By eye (Figure 8).: On these six photos our final model is at least as close to the CCP labels as every other model. YOLO26x-sem breaks the overalls (row 4) into top and pants fragments and puts scarf on a coat collar and bow (rows 1 and 4). SCHP-ATR adds a scarf at the neck (row 1) and creates stray regions from people at the edge of the photo (row 3). FASHN calls the overalls a dress (row 4) and shows shoes as skin. On the long draped skirt (row 3), CCP says dress and every model says skirt, which is the labelling difference described in Section IV-B. The 12-epoch and 30-epoch outputs of our model look alike here; the gain is small, as in Table VII.

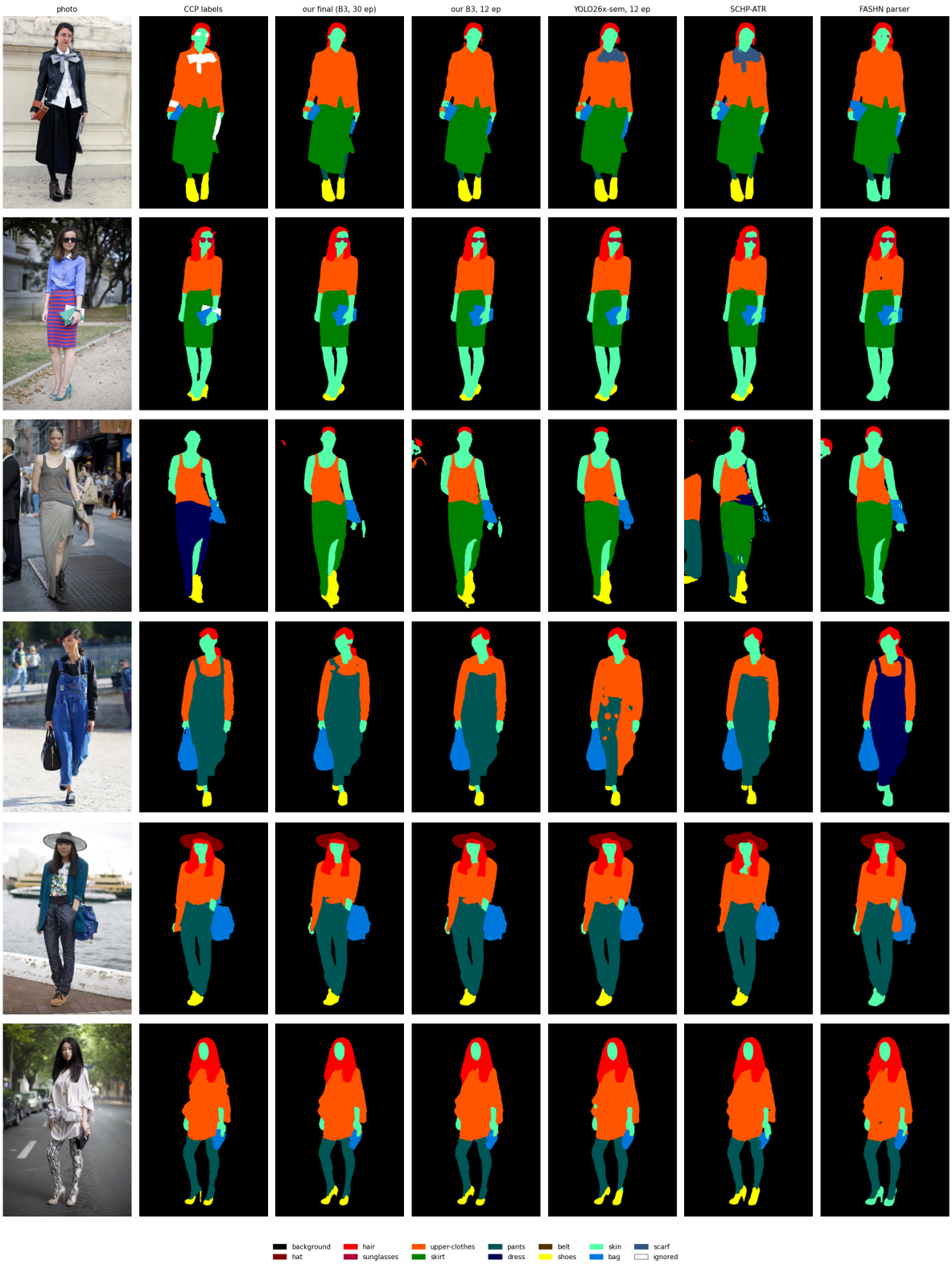


Fig. 8. Side-by-side outputs on six CCP photos: the photo, the CCP labels, our final model (B3, 30 epochs), our B3 at 12 epochs, YOLO26x-sem (12 epochs), SCHP-ATR and the FASHN parser. FASHN has no shoes class, so its shoes appear as skin.

TABLE VII

OUR FINAL SEGFORMER-B3 (30 EPOCHS) AGAINST OTHER MODELS ON CCP (986 PHOTOS NONE WAS TRAINED ON). GPU = TRAINING COST, PLUS SCORING ALL FIVE MODELS AND THE ERROR ANALYSIS ON CCP.

	Ours, B3 30 ep	**Ours, B3 12 ep**	**FASHN**	**YOLO26x-sem**	**SCHP-ATR**
Garment mIoU, 9 classes (no shoes)	**0.660**	0.650	0.641	0.627	0.563
Garment mIoU, 10 classes	**0.676**	0.667	–	0.645	0.584
mIoU, 13 classes	**0.726**	0.718	–	0.698	0.634
upper-clothes / pants	**0.835 / 0.871**	0.829 / 0.855	0.833 / 0.845	0.817 / 0.841	0.779 / 0.747
skirt / dress	**0.401** / 0.475	0.378 / 0.451	0.398 / **0.500**	0.375 / 0.451	0.339 / 0.356
hat / sunglasses / belt	**0.760 / 0.722** / 0.497	0.753 / 0.714 / **0.503**	0.713 / 0.623 / 0.466	0.712 / 0.686 / 0.498	0.652 / 0.565 / 0.388
bag / scarf	**0.786** / 0.592	0.769 / 0.598	0.781 / **0.605**	0.747 / 0.510	0.719 / 0.525
ms per photo (CCP, no flip)	130	116	163	90	132
Gap to ours, 9 garments		−1.0	−1.9	−3.4	−9.7
Training GPU time	6.4 h	3.1 h	not trained here	5.8 h†	not trained here

Scoring all five on CCP, with the error analysis: about 1.5 GPU h† (in Colab).

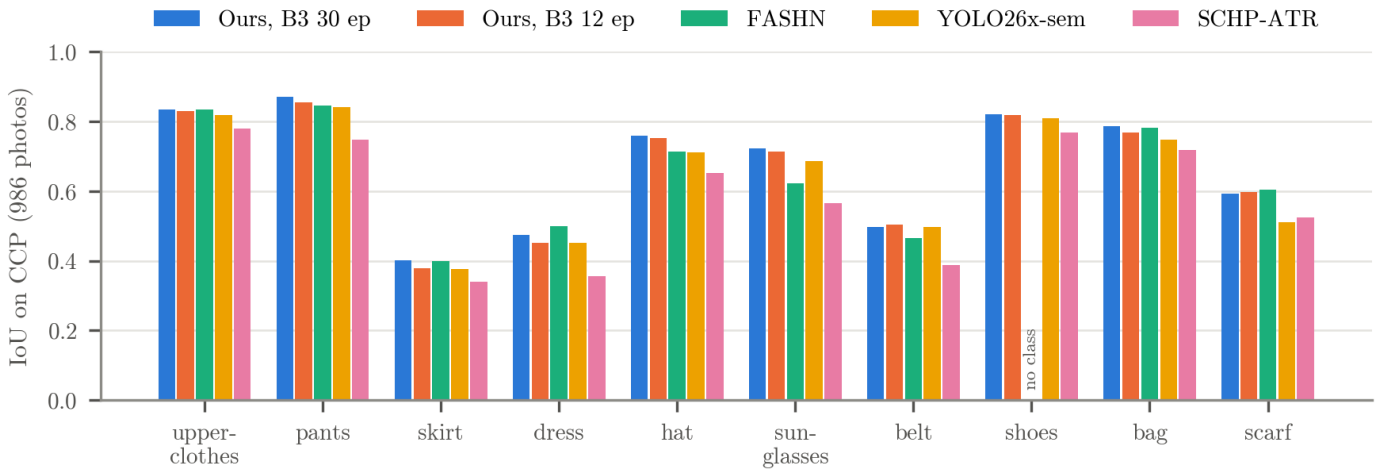


Fig. 9. Per-garment IoU of the five models on the same 986 CCP photos (Table VII). Ours (30 epochs) is highest or level on most garments; FASHN is higher on dress and scarf and has no shoes class.

TABLE VIII

SCHP'S PUBLISHED 82.29 AGAINST OUR NUMBERS.

Same kind of number	**Ours**	**SCHP**
mIoU, all classes, ATR	0.786[a]	0.823[b]
Garment mIoU, ATR	0.741	n/p
mIoU 13 classes, CCP (same photos)	**0.726**	0.634
Garment mIoU, CCP (same photos)	**0.676**	0.584

[a] 15 classes, our split, no flip. [b] 18 classes, official split. n/p: not published.

Our headline 0.744 is a garment-only score. SCHP's 82.29 averages all classes, easy ones included. Like for like on ATR (Table VIII), the gap is about 3.7 points, so SCHP may be better on ATR itself. On new photos the order reverses: SCHP drops from 82 to 63, ours from 79 to 73 (Figure 14). The SCHP weights used are a community repackaging whose reproduction of 82.29 was not verified.

D. Changes tried after the model was chosen

After selection, every change was judged on validation by a rule written before it ran. Table IX lists them with their GPU cost. Figure 15 shows each change against the +0.5 point it needed.

V. DISCUSSION

A. Strengths

Each strength below gives the measured result and the reason it is expected, with the source for the reason.

- **Clothes are reliably separated from the person and background:** binary IoU 0.930 on ATR and 0.947 on CCP. *Why:* SegFormer's overlapping patch embedding keeps local continuity around patch borders and works on fine 4×4-scale patches, unlike the coarse 16×16 patches of a plain ViT [29], [38]. Deciding "clothing or not" also depends on large, high-contrast regions rather than on garment type, which is why it carries over to CCP unchanged.
- **Large garments and body parts are accurate:** pants 0.85, upper-clothes 0.83, legs 0.89, face 0.86; and large-enough accessories: bag 0.82, hat 0.79. *Why:* these classes cover many pixels in almost every training photo. The Lovász term is also scale-invariant, which gives small objects appropriate weight, and was reported to score better on IoU

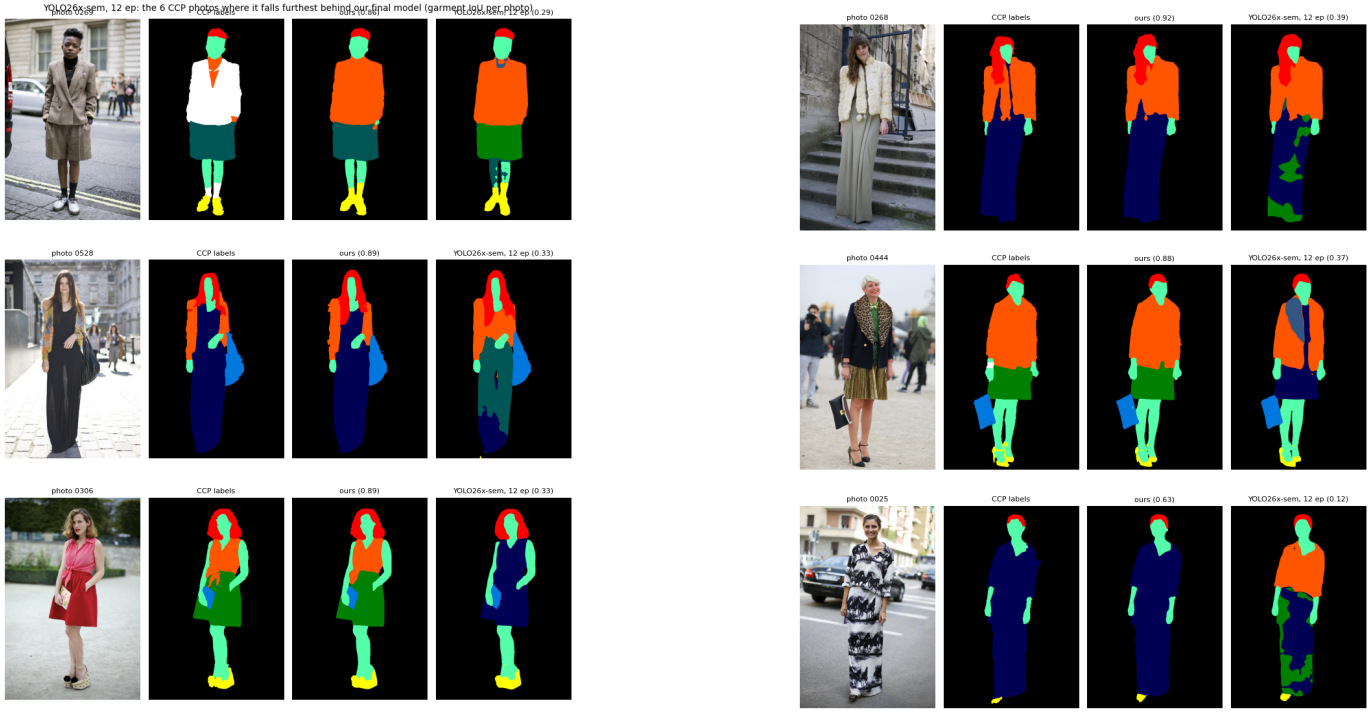


Fig. 10. The six CCP photos where YOLO26x-sem (12 epochs) falls furthest behind our final model. Garment IoU per photo is in each title: ours 0.63–0.92, YOLO26x-sem 0.12–0.39.

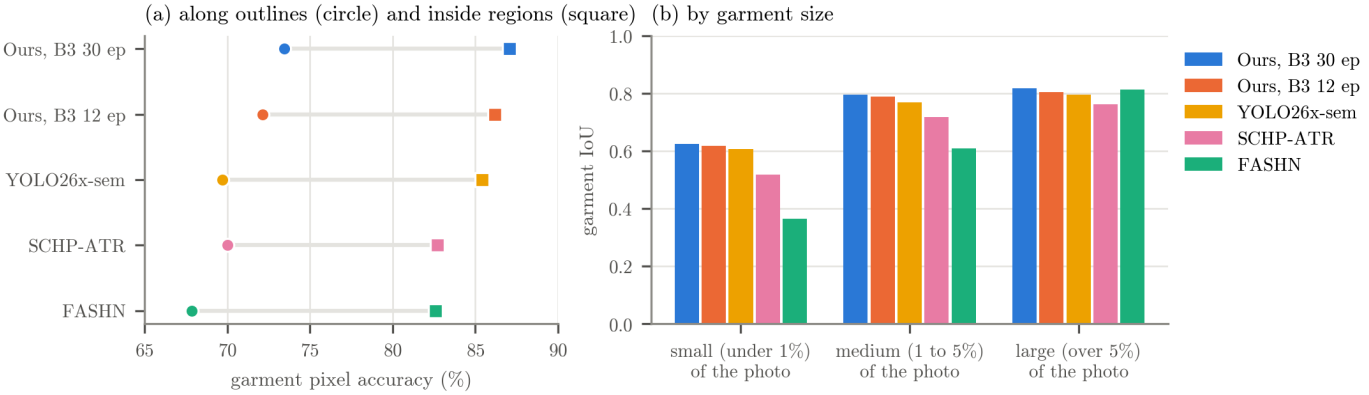


Fig. 11. Where the errors are, on CCP. (a) Garment pixel accuracy along outlines (circles) and inside regions (squares): every model loses more along outlines, YOLO26x-sem most. (b) Garment IoU by how much of the photo the garment covers: SCHP-ATR and FASHN lose most on small garments. FASHN's pooled figures count its missing shoes class as errors, so they understate it.

than plain cross-entropy [33]. This loss was not ablated here, so it is a likely reason, not a proven one.

- **It generalises:** validation and test agree within 0.1 point, and it beats YOLO26x-sem, SCHP and FASHN on unseen photos. *Why:* near-duplicate photos across splits inflate test scores [3], and the 110 such images were moved to training, so the test score is not leaked. Training used blur, noise and JPEG augmentation, and SegFormer (the B5 model) was shown to be robust to 16 corruption types on Cityscapes-C [29]; B3 shares the same encoder design.
- **Person outlines are right even in the worst cases;** the errors concern garment type. *Why:* the all-MLP decoder predicts at 1/4 of the input, against 1/8 for the YOLO26x-sem head, and YOLO's outline loss was about twice its inside loss (Section IV-C). Overlapping patches also keep edges continuous [29].
- **Any photo size and orientation** is handled, including EXIF rotation, transparent and grey-scale photos. *Why:* the encoder has no positional encoding to interpolate. Mix-FFN uses a 3×3 depth-wise convolution that leaks position through zero padding, so accuracy does not fall when the test resolution differs from training [29], [38]. EXIF and colour-mode handling are done in the pipeline (Section III-D).
- **Practical:** 56 ms per photo on an L4, inference fits a T4, and training survives interrupted sessions. *Why:* efficient



Fig. 12. Sample outputs: three of the six CCP photos where SCHP-ATR (left) and FASHN (right) fall furthest behind our final model (photo, CCP labels, ours, the other model; garment IoU per photo in each title). SCHP-ATR splits dresses into top and skirt; FASHN turns a top with a skirt into a dress and shoes into skin.

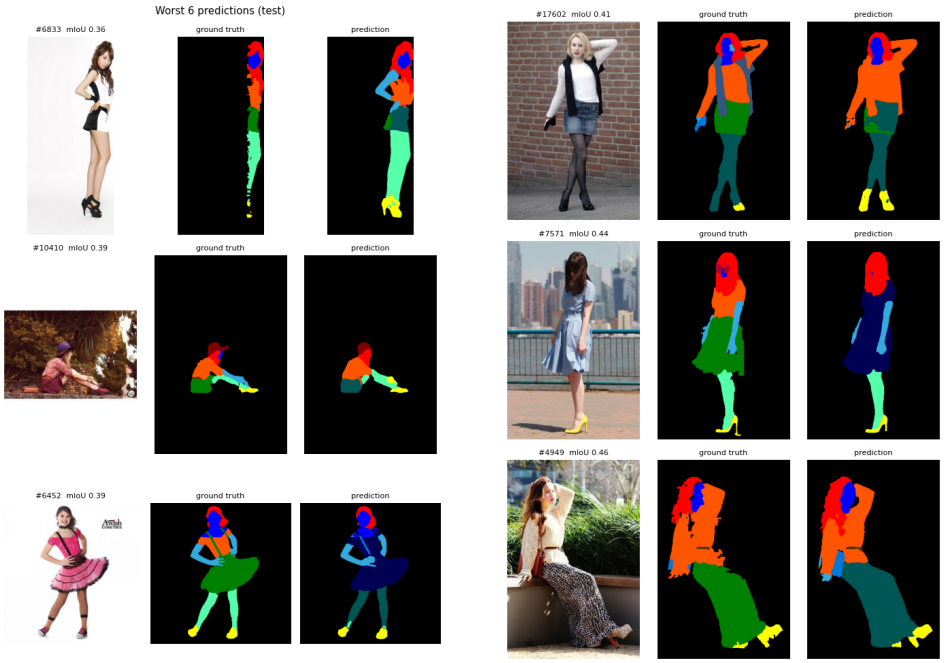


Fig. 13. Worst test predictions (per-image mIoU 0.36–0.46): a faulty label, a small sitting person, a top with a tutu taken for a dress, layered clothing, and a long skirt seen as pants.

self-attention shortens the key and value sequences, cutting the cost from $O(N^2)$ to $O(N^2/R)$, with $R = 64, 16, 4, 1$ in the four stages [29], and the decoder is light: 3.2 M of the 47.2 M parameters. Resumable training comes from the

TABLE IX
CHANGES TRIED AFTER THE MODEL WAS CHOSEN, WITH THEIR GPU COST.

Change	Result (validation)	GPU	Decision
Edge branch (CE2P/SCHP) [5], 12 ep	0.7212 vs 0.7302; belt, scarf lower; +15% time	3.5 h	Rejected (≥0.7353 needed)
ADE20K-pretrained start, 12 ep	0.7145 vs 0.7302; behind from epoch 1	3.1 h†	Rejected
Two-encoder B3 + ConvNeXt-T, 83 M [31], 12 ep, then continued	+0.12; dress −2.4, skirt −1.7; 43% slower. Continued: 0.7067 at epoch 17 (B3: 0.7294), validation loss rising	4.1 h + 2.2 h†	Rejected; stopped at epoch 17
Ensemble B3 + UPerNet (no training)	−0.15; scarf −4.1	1.2 h†	Rejected
Multi-scale averaging ×0.75/1/1.25 + flip	+0.07 mIoU; main garments lower; 2.8× slower	1.2 h†	Rejected
Cyclic fine-tune with weight averaging, 9 ep	0.7442 vs 0.7441	2.5 h†	Rejected (stable point)
Fine-tune at 768×512, 3 ep	+0.26; scarf +2.8, sunglasses +1.1, but skirt −3.8	2.3 h†	Rejected (core −1)
Half learning rate, 12 ep	0.7179 vs 0.7302	3.1 h†	Rejected
Double learning rate, 12 ep	0.7348 (+0.46); dress −1.1	3.1 h†	Rejected (just under +0.5)
SegFormer-B4 (larger), 12 ep	0.7231 vs 0.7303	3.6 h†	Not better
No-augmentation ablation (B2)	Ran out of memory on a T4 after epoch 1	1.1 h† (T4)	No result
15 vs 18 classes	Planned; dropped for time	–	Not compared

TABLE X
NEXT STEPS, THEIR EXPECTED VALUE AND COST, AND WHAT STOPPED THEM.

Next step	Expected value	GPU	What stopped it
ATR + DeepFashion-MultiModal, partial-label loss [39]	Possible gain on dress vs skirt; none for scarf	4.8–6.8 h†	Fresh control, equal steps and a bootstrap test needed before the deadline; non-commercial licence
Larger encoders (EoMT ViT-L, Sapiens)	Better on other benchmarks	≥15.8–30.8 h†	6–25× our size; may not fit 22 GB
SegNeXt (MSCAN)	Strongest untested equal-size candidate	3.3 h† per run	Old code base
Second and third seeds	Trust 1-point differences	11.8 h†	Budget
Learning rate ×1.5–2, 768×512 at full length	Small positive signs	6.4–8.8 h† each	Budget
ModaNet thin-accessory data	Belt, sunglasses, scarf	–	Photos no longer available
Label cleaning (SCHP self-correction)	Largest gain for ambiguous labels	≥30.8 h†	150 epochs + cycles

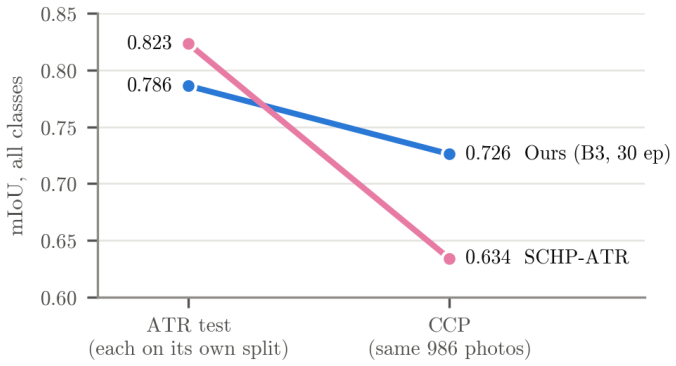


Fig. 14. mIoU over all classes on ATR (each model on its own test split, so not exactly comparable) and on the same 986 CCP photos (Table VIII). SCHP-ATR drops by 19 points on new photos, ours by 6.

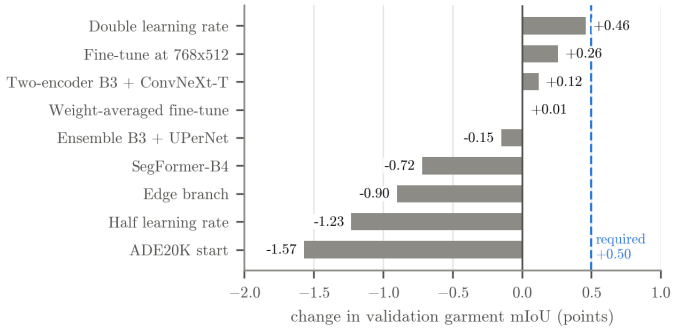


Fig. 15. Change in validation garment mIoU of each change tried after selection, against the model it was compared with (Table IX). None reached the +0.5 point required; multi-scale averaging is left out because it was judged on mIoU.

atomic saves (Section III-C).

B. Drawbacks and failure cases

- **Garment type is the main failure:** dress against top-plus-skirt, and long skirts against trousers when the person sits.
- **Thin items are weak:** belt 0.52, sunglasses 0.69, scarf 0.70.
- **Layers are merged:** with no coat class, a jacket and the top under it form one region.
- **Garments outside ATR's classes are split inconsistently:** a gown becomes upper-clothes plus pants in one photo and a dress in another.
- **Inherited ATR conventions:** flat objects held in the hands become bags, tights become pants, the neck is labelled face, and clear glasses are missed.
- **Some measured error is label error:** one of the six worst test cases is a faulty label (Figure 13).

C. Limitations

a) Capture conditions.: The model should be used with photos like ATR's. 28 real photos unlike ATR's (phone, street, group selfies, screenshots, stage light) were judged by eye, which shows *where* it fails, not how often. Measuring this on labelled photos is left for future work. Table XI gives the expected behaviour by capture condition.

TABLE XI
EXPECTED BEHAVIOUR BY CAPTURE CONDITION.

Condition	Works best	Expect worse
People	One	Several share one mask
Framing	Person fills most of the frame	Small person; body cut off
Pose	Standing, upright	Sitting, lying; tilt > 15°
Resolution	Person ≥400 px tall	Much smaller
Lighting	Even light	Very dark; strong backlight
Background	Person stands out	Mannequins, posters
Clothing	Common garments	Layers, gowns, costumes
Occlusion	None or small	Large objects, other people

In the stress test:

- **robust:** empty streets gave no false person, and screenshots and stage light still gave plausible results;
- **failed:** distant people were coarse or missed.

VI. GPU TIME USED AND NEXT STEPS

Table XII gives the GPU time used for each part of the work, and Table X the next steps, their expected value and cost, and what stopped them.

TABLE XII
GPU TIME USED (L4 UNLESS STATED).

Work	GPU h
Bake-off (5 models, rounds 1–2)	9.1
B3 variants (edge, ADE20K, 2 learning rates)	10.4
Two-encoder network (12 + 5 epochs)	5.5
YOLO26x-sem (12 epochs, one crashed start)	5.8†
Final model, 30 epochs	6.4
Fine-tunes (weight averaging, 768×512)	4.0†
Evaluations, CCP, ensemble, multi-scale	2.9†
Data preparation, benchmarks, T4 ablation	1.6†
Total	**≈46**

About 16 h of training were measured directly; the rest (marked † in the tables) are estimated from measured per-epoch speeds. Each entry includes about 50 min per run for setup and evaluation (estimate).

REFERENCES

[1] X. Liang et al., "Deep Human Parsing with Active Template Regression," TPAMI 2015.
[2] ATR on Hugging Face. https://huggingface.co/datasets/mattmdjaga/human_parsing_dataset
[3] B. Barz, J. Denzler, "Do We Train on Test Data?," 2019. https://arxiv.org/abs/1902.00423
[4] K. Gong et al., "Graphonomy," CVPR 2019. https://arxiv.org/abs/1904.04536
[5] T. Ruan et al., "Devil in the Details (CE2P)," AAAI 2019. https://arxiv.org/abs/1809.05996
[6] W. Yang et al., "Clothing Co-Parsing," CVPR 2014. https://github.com/bearpaw/clothing-co-parsing
[7] K. Gong et al., "Look into Person," CVPR 2017. https://arxiv.org/abs/1703.05446
[8] FASHN blog: dataset problems; open-sourcing the human parser. https://fashn.ai/blog
[9] K. Gong et al., "Instance-level Human Parsing via Part Grouping Network," ECCV 2018. https://arxiv.org/abs/1808.00157
[10] J. Zhao et al., "Understanding Humans in Crowded Scenes," 2018. https://arxiv.org/abs/1804.03287
[11] Y. Ge et al., "DeepFashion2," CVPR 2019. https://github.com/switchablenorms/DeepFashion2
[12] S. Zheng et al., "ModaNet," ACM MM 2018. https://github.com/eBay/modanet
[13] M. Jia et al., "Fashionpedia," ECCV 2020. https://fashionpedia.github.io/home/
[14] Fashionpedia terms. https://fashionpedia.github.io/home/data_license.html
[15] J. Fu et al., "StyleGAN-Human," 2022. https://arxiv.org/abs/2204.11823
[16] D. Morelli et al., "Dress Code," ECCV 2022. https://github.com/aimagelab/dress-code
[17] W. Wang et al., "4D-DRESS," CVPR 2024. https://github.com/eth-ait/4d-dress
[18] Z. Yang et al., "SynBody," ICCV 2023.
[19] P. Li et al., "Self-Correction for Human Parsing," TPAMI 2020. https://arxiv.org/abs/1910.09777
[20] Y. Jiang et al., "Text2Human / DeepFashion-MultiModal," 2022. https://github.com/yumingj/DeepFashion-MultiModal
[21] T. Kerssies et al., "Your ViT is Secretly an Image Segmentation Model," CVPR 2025. https://github.com/tue-mps/eomt
[22] R. Khirodkar et al., "Sapiens," ECCV 2024. https://huggingface.co/facebook/sapiens
[23] "SAM 3: Segment Anything with Concepts," 2025. https://arxiv.org/abs/2511.16719
[24] M.-H. Guo et al., "SegNeXt," NeurIPS 2022. https://arxiv.org/abs/2209.08575
[25] Y. Yuan et al., "Object-Contextual Representations," ECCV 2020. https://arxiv.org/abs/1909.11065
[26] M2FP. https://github.com/soeaver/M2FP
[27] Ultralytics segmentation. https://docs.ultralytics.com/tasks/segmentation/
[28] SCHNet https://arxiv.org/abs/2503.22237; Spectrum https://arxiv.org/abs/2508.06032; CDGNet https://arxiv.org/abs/2111.14173.
[29] E. Xie et al., "SegFormer," NeurIPS 2021. https://arxiv.org/abs/2105.15203
[30] SCHP code and results. https://github.com/GoGoDuck912/Self-Correction-Human-Parsing
[31] Z. Peng et al., "Conformer," ICCV 2021. https://arxiv.org/abs/2105.03889
[32] Ultralytics YOLO26. https://docs.ultralytics.com/models/yolo26/
[33] M. Berman et al., "The Lovász-Softmax Loss," CVPR 2018. https://arxiv.org/abs/1705.08790
[34] Official SegFormer code. https://github.com/NVlabs/SegFormer
[35] B. Cheng et al., "Boundary IoU," CVPR 2021. https://arxiv.org/abs/2103.16562
[36] SCHP-ATR for Transformers. https://huggingface.co/pirocheto/schp-atr-18
[37] FASHN Human Parser. https://huggingface.co/fashn-ai/fashn-human-parser
[38] H. Thisanke et al., "Semantic Segmentation using Vision Transformers: A survey," 2023. https://arxiv.org/abs/2305.03273
[39] P. Bevandić et al., "Multi-domain Semantic Segmentation with Overlapping Labels," WACV 2022. https://arxiv.org/abs/2108.11224